

1/20/2025	9 AM - 10 AM
-----------	--------------

Advanced DML (Data Manipulation Language) refers to more complex SQL operations that go beyond simple SELECT, INSERT, UPDATE, and DELETE statements. These operations often involve working with subqueries, joins, transactions, and other advanced features of SQL that allow for more sophisticated data manipulation.

Key Advanced DML Concepts:

1. **Subqueries:** A subquery is a query within another query, used to return a result set that can be used in the main query.
2. **Joins:** Joins combine rows from two or more tables based on a related column. Common joins include INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL JOIN.
3. **Transactions:** Transactions allow you to execute multiple DML statements as a single unit, ensuring consistency and atomicity of operations.
4. **Set Operations:** These operations allow you to combine results from multiple queries, like UNION, INTERSECT, and EXCEPT.
5. **CASE Statements:** A conditional expression that allows you to perform IF-ELSE logic in SQL queries.
6. **MERGE (Upsert):** A command that allows you to perform an INSERT, UPDATE, or DELETE based on certain conditions (useful for synchronizing tables).
7. **Top and OFFSET:** These clauses are used to restrict the number of records returned by a query.

Select top 10 * from table name.

Advanced DML Examples:

1. Subqueries:

A subquery is a query within another query. It can be used in the SELECT, INSERT, UPDATE, or DELETE statements.

- **Subquery in SELECT:**

sql

SELECT Name, Age

FROM Employees

WHERE Department IN (SELECT Department FROM Departments WHERE Location = 'New York');

- **Subquery in UPDATE:**

sql

UPDATE Employees

SET Age = 35

WHERE ID = (SELECT ID FROM Employees WHERE Name = 'John Doe');

- **Subquery in DELETE:**

sql

DELETE FROM Employees

WHERE ID IN (SELECT ID FROM Employees WHERE Department = 'HR');

3. Transactions:

Transactions allow you to execute multiple DML commands together, ensuring that they are treated as a single unit.

- **Begin a Transaction:**

sql

BEGIN TRANSACTION;

- **Commit the Transaction:** Saves the changes made during the transaction.

sql

COMMIT;

- **Rollback the Transaction:** Rolls back the transaction if there is an error.

sql

ROLLBACK;

Example:

sql

BEGIN TRANSACTION;

UPDATE Employees

SET Age = 40

WHERE ID = 1;

DELETE FROM Employees

WHERE ID = 2;

ROLLBACK;

4. Set Operations:

These operations allow you to combine the results of multiple queries.

- **UNION:** Combines the result sets of two queries, removing duplicates.

sql

SELECT Name FROM Employees

UNION

SELECT Name FROM Managers;

- **INTERSECT:** Returns only the common records between two queries.

sql

```
SELECT Name FROM Employees
```

```
INTERSECT
```

```
SELECT Name FROM Managers;
```

- **EXCEPT:** Returns rows from the first query that aren't in the second.

sql

```
SELECT Name FROM Employees
```

```
EXCEPT
```

```
SELECT Name FROM Managers;
```

1/20/2025	10 AM - 11 AM
-----------	---------------

5. CASE Statement:

The CASE statement is used to perform conditional logic inside queries.

sql

```
SELECT Name, Age,
```

```
    CASE
```

```
        WHEN Age < 18 THEN 'Minor'
```

```
        WHEN Age >= 18 AND Age < 65 THEN 'Adult'
```

```
        ELSE 'Senior'
```

```
    END AS AgeGroup
```

```
FROM Employees;
```

6. MERGE (Upsert):

The MERGE statement allows you to insert, update, or delete records depending on whether they exist.

Example:

sql

MERGE INTO Employees AS target

USING (SELECT 1 AS ID, 'John Doe' AS Name, 'HR' AS Department) AS source

ON target.ID = source.ID

WHEN MATCHED THEN

UPDATE SET target.Name = source.Name, target.Department = source.Department

WHEN NOT MATCHED THEN

INSERT (ID, Name, Department)

VALUES (source.ID, source.Name, source.Department)

WHEN NOT MATCHED BY SOURCE THEN

DELETE;

7. Top and OFFSET:

LIMIT restricts the number of rows returned, and OFFSET specifies how many rows to skip before starting to return results.

sql

SELECT top 10 Name, Age

FROM Employees

OFFSET 20;

Notes:

- **Transactions** ensure atomicity, meaning either all operations are completed, or none are if an error occurs.
- **Set operations** (UNION, INTERSECT, EXCEPT) are useful for combining results from multiple queries.
- **Joins** allow you to work with related data in multiple tables.

- **Subqueries** provide a way to embed a query within another query, enhancing flexibility.
- **MERGE** is powerful for synchronizing data between tables, such as when updating records or inserting new ones based on conditions.

1/20/2025	11 AM - 12 PM
-----------	---------------

Advanced SQL Concepts encompass a range of sophisticated techniques and optimizations that go beyond basic querying. These concepts help with more complex data analysis, performance optimization, and managing large databases. Here are several key advanced SQL topics:

1. Window Functions:

Window functions allow you to perform calculations across a set of table rows related to the current row without collapsing the rows into a single output. They're very useful for ranking, running totals, moving averages, etc.

- **Syntax:**

sql

SELECT column1, column2,

 WINDOW_FUNCTION() OVER (PARTITION BY column3 ORDER BY column4)

FROM table_name;

- **Examples:**

- **ROW_NUMBER():** Assigns a unique number to each row within the result set, based on a given order.

sql

SELECT Name, Salary,

 ROW_NUMBER() OVER (ORDER BY Salary DESC) AS RowNum

FROM Employees;

- **RANK():** Similar to ROW_NUMBER(), but gives the same rank to identical rows.

sql

SELECT Name, Salary,

RANK() OVER (ORDER BY Salary DESC) AS Rank

FROM Employees;

- **SUM() OVER():** Calculate a cumulative sum across a partition of data.

sql

SELECT Name, Salary,

SUM(Salary) OVER (ORDER BY Salary) AS RunningTotal

FROM Employees;

Cumulative sum

2. Common Table Expressions (CTE):

CTEs are temporary result sets that can be referenced within a SELECT, INSERT, UPDATE, or DELETE statement. They improve readability and structure, especially for recursive queries.

- **Syntax:**

sql

WITH CTE_Name AS (

SELECT column1, column2

FROM table_name

WHERE condition

)

SELECT column1, column2

FROM CTE_Name;

- **Example:**

sql

WITH DepartmentAvg AS (

 SELECT Department, AVG(Salary) AS AvgSalary

 FROM Employees

 GROUP BY Department

)

SELECT e.Name, e.Department, e.Salary, d.AvgSalary

FROM Employees e

JOIN DepartmentAvg d ON e.Department = d.Department;

3. Recursive Queries:

A recursive query is useful when dealing with hierarchical or tree-structured data (e.g., organizational charts, category structures).

- **Syntax:**

sql

WITH CTE_Name AS (

 -- Base case: Select the first level of data

 SELECT column1, column2

 FROM table_name

 WHERE condition

 UNION ALL

 -- Recursive case: Select subsequent levels of data

 SELECT column1, column2

 FROM table_name t

 JOIN CTE_Name cte ON t.column1 = cte.column1

)


```
SELECT * FROM CTE_Name;
```

- **Example:**

sql

-----SQL

```
WITH EmployeeHierarchy AS (
```

```
    SELECT EmployeeID, EmployeeName, ManagerID
```

```
    FROM Employees
```

```
    WHERE ManagerID IS NULL
```

```
UNION ALL
```

```
    SELECT e.EmployeeID, e.EmployeeName, e.ManagerID
```

```
    FROM Employees e
```

```
    JOIN EmployeeHierarchy eh ON e.ManagerID = eh.EmployeeID
```

```
)
```

```
SELECT * FROM EmployeeHierarchy;
```

12. Data Integrity Constraints:

Data integrity constraints, such as **primary keys**, **foreign keys**, **unique constraints**, **check constraints**, and **default values**, are used to enforce rules on data in the database to ensure consistency and reliability.

- **Example:**

sql

```
ALTER TABLE Employees
```

```
ADD CONSTRAINT fk_department
```

```
FOREIGN KEY (DepartmentID)
```

```
REFERENCES Departments(DepartmentID);
```

1/20/2025	12PM - 1 PM
-----------	-------------

Advanced SQL Concepts:

SQL JOIN:

A **JOIN** in SQL is used to combine rows from two or more tables based on a related column between them. The SQL JOIN clause is used to extract data from these multiple tables based on some condition, usually related to a **primary key** and a **foreign key**.

There are several types of joins in SQL, and each type serves a different purpose for retrieving data from the tables.

Types of SQL Joins:

1. INNER JOIN:

- **Definition:** The INNER JOIN returns rows when there is a match in both tables.
- **Usage:** This is the most commonly used type of join. It only returns rows that have matching values in both tables.
- **Syntax:**

sql

```
SELECT a.col1, a.col2, a.col3, b.col1, b.col2
```

```
FROM table1 a
```

```
INNER JOIN table2 b
```

```
ON a.col1 = b.col1;
```

- **Example:**

sql

```
SELECT Employees.Name, Departments.DepartmentName
```

```
FROM Employees
```

```
INNER JOIN Departments
```

```
ON Employees.DepartmentID = Departments.DepartmentID;
```

2. LEFT JOIN (or LEFT OUTER JOIN):

- **Definition:** The LEFT JOIN returns all rows from the left table and the matching rows from the right table. If there is no match, the result is NULL for the right table.
- **Usage:** This is used when you want to return all records from the left table even if there is no corresponding match in the right table.
- **Syntax:**

sql

```
SELECT column_names  
FROM table1  
LEFT JOIN table2  
ON table1.column = table2.column;
```

- **Example:**

sql

```
SELECT Employees.Name, Departments.DepartmentName  
FROM Employees  
LEFT JOIN Departments  
ON Employees.DepartmentID = Departments.DepartmentID;
```

In this case, even if some employees do not belong to any department, they will still appear in the results, with NULL values for the department name.

3. RIGHT JOIN (or RIGHT OUTER JOIN):

- **Definition:** The RIGHT JOIN returns all rows from the right table and the matching rows from the left table. If there is no match, the result is NULL for the left table.
- **Usage:** This is used when you want to return all records from the right table, even if there is no corresponding match in the left table.
- **Syntax:**

sql

```
SELECT column_names  
FROM table1  
RIGHT JOIN table2  
ON table1.column = table2.column;
```

- **Example:**

sql

```
SELECT Employees.Name, Departments.DepartmentName  
FROM Employees  
RIGHT JOIN Departments  
ON Employees.DepartmentID = Departments.DepartmentID;
```

In this case, even if some departments have no employees, they will still appear in the result, with NULL values for the employee names.

4. **FULL JOIN (or FULL OUTER JOIN):**

- **Definition:** The FULL JOIN returns all rows when there is a match in either the left or the right table. If there is no match, the result is NULL from the table that does not have the matching record.
- **Usage:** This is useful when you want to retrieve all rows from both tables, including non-matching rows.
- **Syntax:**

sql

```
SELECT column_names  
FROM table1  
FULL JOIN table2  
ON table1.column = table2.column;
```

- **Example:**

sql

```
SELECT Employees.Name, Departments.DepartmentName
```

```
FROM Employees
```

```
FULL JOIN Departments
```

```
ON Employees.DepartmentID = Departments.DepartmentID;
```

In this case, if there are departments with no employees or employees with no department, they will still appear in the result, with NULL values where necessary.

5. **CROSS JOIN:**

- **Definition:** The CROSS JOIN returns the Cartesian product of both tables, meaning it will return all possible combinations of rows from both tables.
- **Usage:** This is useful when you want to combine every row from one table with every row from another table, resulting in a potentially very large dataset.
- **Syntax:**

sql

```
SELECT column_names
```

```
FROM table1
```

```
CROSS JOIN table2;
```

- **Example:**

sql

```
SELECT Employees.Name, Departments.DepartmentName
```

```
FROM Employees
```

```
CROSS JOIN Departments;
```

In this case, if there are 3 employees and 4 departments, the result will contain 12 rows (3 * 4), where each employee is combined with each department.

6. SELF JOIN:

- **Definition:** A SELF JOIN is a join where a table is joined with itself. This is useful when you have a hierarchical structure (e.g., employees and managers) in the same table and need to relate records to each other.
- **Usage:** Self joins require table aliases to differentiate between the multiple instances of the same table.
- **Syntax:**

sql

```
SELECT a.column1, b.column2
```

```
FROM table a
```

```
JOIN table b
```

```
ON a.column = b.column;
```

- **Example:**

sql

-----SQL

```
select a.EmployeeName Emp, b.EmployeeName Mgr, a.Department Dpt from  
Employees a
```

```
left join Employees b
```

```
on b.EmployeeId=a.ManagerID
```

In this case, we are joining the Employees table with itself to show each employee and their respective manager.

Key Points to Remember:

- **INNER JOIN** returns only matching rows from both tables.
- **LEFT JOIN** returns all rows from the left table, with matching rows from the right table (or NULL if no match).
- **RIGHT JOIN** returns all rows from the right table, with matching rows from the left table (or NULL if no match).

- **FULL JOIN** returns all rows when there is a match in either the left or the right table.
 - **CROSS JOIN** returns a Cartesian product of both tables (every combination of rows).
 - **SELF JOIN** is used when a table is joined with itself, usually for hierarchical data.
-

Use Case Example:

Assume we have two tables:

1. Employees Table:

EmployeeID	Name	DepartmentID
1	John Smith	101
2	Jane Doe	102
3	Sam Brown	103

2. Departments Table:

DepartmentID	DepartmentName
101	HR
102	IT
103	Sales
104	Marketing

INNER JOIN Example:

sql

```
SELECT Employees.Name, Departments.DepartmentName
FROM Employees
INNER JOIN Departments
ON Employees.DepartmentID = Departments.DepartmentID;
```

Result:

Name	DepartmentName
------	----------------

John Smith	HR
------------	----

Jane Doe	IT
----------	----

Sam Brown	Sales
-----------	-------

LEFT JOIN Example:

sql

```
SELECT Employees.Name, Departments.DepartmentName
```

```
FROM Employees
```

```
LEFT JOIN Departments
```

```
ON Employees.DepartmentID = Departments.DepartmentID;
```

Result:

Name	DepartmentName
------	----------------

John Smith	HR
------------	----

Jane Doe	IT
----------	----

Sam Brown	Sales
-----------	-------

In the above cases, if there were any employees without a department (e.g., DepartmentID of NULL), they would still appear in the result with NULL for the department name.

2. Joins:

Joins combine data from multiple tables based on a related column.

- **INNER JOIN:** Returns rows that have matching values in both tables.

sql

```
SELECT Employees.Name, Departments.DepartmentName
```

```
FROM Employees
```


INNER JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID;

- **LEFT JOIN:** Returns all rows from the left table and matched rows from the right table, and NULL if no match.

sql

SELECT Employees.Name, Departments.DepartmentName

FROM Employees

LEFT JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID;

- **RIGHT JOIN:** Returns all rows from the right table and matched rows from the left table, and NULL if no match.

sql

SELECT Employees.Name, Departments.DepartmentName

FROM Employees

RIGHT JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID;

- **FULL JOIN:** Returns all rows when there is a match in either left or right table.

sql

SELECT Employees.Name, Departments.DepartmentName

FROM Employees

FULL JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID;

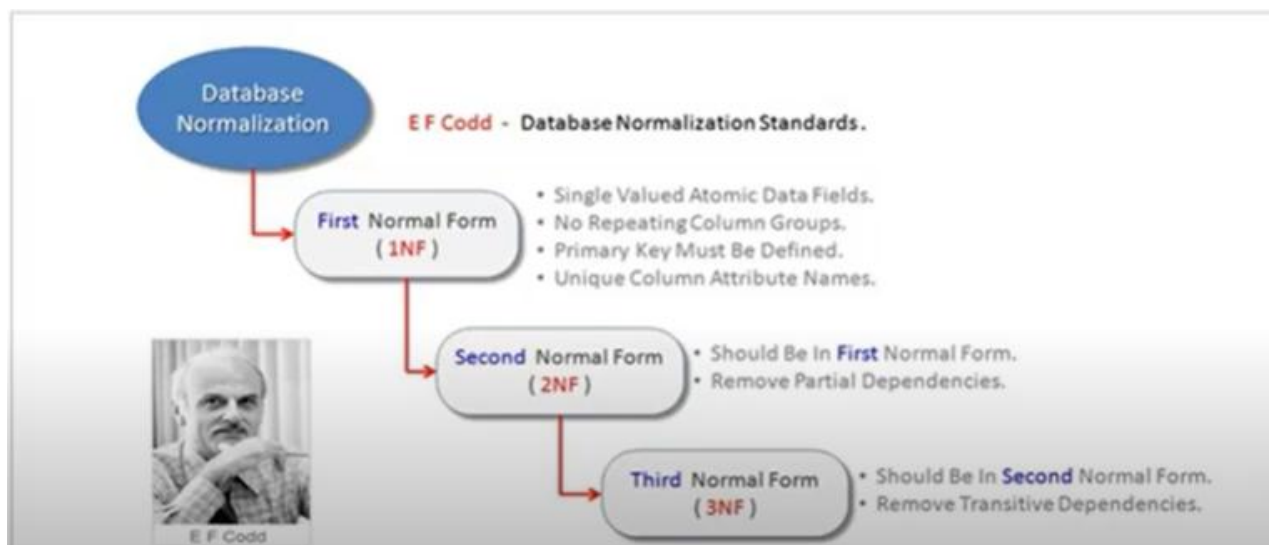
1/20/2025	2 PM - 3 PM
-----------	-------------

5. Database Normalization:

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. There are several normal forms (1NF, 2NF, 3NF, BCNF, etc.), each with specific rules for how data should be structured.

- **Example:**

- 1NF: Ensures that each column contains atomic (indivisible) values and each record is unique. No repetitive column, Primary key.
- 2NF: Ensures that the table is in 1NF and all non-key attributes are fully functionally dependent on the primary key.
- 3NF: Ensures that the table is in 2NF and that all non-key attributes are non-transitively dependent on the primary key.



Examples of 1NF, 2NF, and 3NF

Database normalization is quite technical, but we will illustrate each of the normal forms with examples.

Imagine we're building a restaurant management application. That application needs to store data about the company's employees and it starts out by creating the following table of employees:

Not in First Normal Form

employee_id	name	job_code	job	state_code	home_state
E001	Alice	J01,J02	Chef , Waiter	26	Michigan
E002	Bob	J02 ,J03	Waiter , Bartender	56	Wyoming
E003	Alice	J01	Chef	56	Wyoming

OR

employee_id	name	<u>job_code</u>	<u>job_code2</u>	<u>job</u>	<u>job2</u>	state_code	home_state
E001	Alice	J01	J02	Chef	Waiter	26	Michigan
E002	Bob	J02	J03	Waiter	Bartender	56	Wyoming
E003	Alice	J01		Chef		56	Wyoming

Make it atomic value.

PK					
employee_id	name	job_code	job	state_code	home_state
E001	Alice	J01	Chef	26	Michigan
E001	Alice	J02	Waiter	26	Michigan
E002	Bob	J02	Waiter	56	Wyoming
E002	Bob	J03	Bartender	56	Wyoming
E003	Alice	J01	Chef	56	Wyoming

Primary key as composite key (employee_id, job_code), We can not update the name here.

All the entries are atomic and there is a composite primary key (employee_id, job_code) so the table is in the **first normal form (1NF)**.

But even if you only know someone's employee_id, then you can determine their name, home_state, and state_code (because they should be the same person). This means name, home_state, and state_code are dependent on employee_id (a part of primary composite key). So, the table is not in **2NF**. We should separate them to a different table to make it 2NF.

Example of Second Normal Form (2NF)

employee_roles **Table**

employee_roles

FK	FK
employee_id	job_code
E001	J01
E001	J02
E002	J02
E002	J03
E003	J01

employees Table

PK			
employee_id	name	state_code	home_state
E001	Alice	26	Michigan
E002	Bob	56	Wyoming
E003	Alice	56	Wyoming

jobs table

PK	
job_code	job
J01	Chef
J02	Waiter
J03	Bartender

home_state is now dependent on state_code. So, if you know the state_code, then you can find the home_state value.

To take this a step further, we should separate them again to a different table to make it 3NF.

Example of Third Normal Form (3NF)

employee_roles Table

Transactional Table	FK
FK	
employee_id	job_code
E001	J01
E001	J02
E002	J02
E002	J03
E003	J01

employees Table

PK		
employee_id	name	state_code
E001	Alice	26
E002	Bob	56
E003	Alice	56

jobs Table

PK	
job_code	job
J01	Chef
J02	Waiter
J03	Bartender

states Table

PK	
state_code	home_state
26	Michigan
56	Wyoming

Now our database is in 3NF.

1/20/2025	3 PM - 4 PM
-----------	-------------

6. Denormalization:

Denormalization is the opposite of normalization and involves combining tables to reduce the complexity of joins. It is used for optimization, especially when you need to improve read performance at the expense of write performance and storage efficiency.

- **Example:** Instead of normalizing employee information across several tables (employees, departments, roles), you might combine them into a single table for faster query performance.

7. Triggers:

A trigger is a special type of stored procedure that automatically runs when specific events occur in the database, such as INSERT, UPDATE, or DELETE on a table.

- **Syntax:**

sql

```
CREATE TRIGGER trigger_name
```

```
AFTER INSERT ON table_name
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    -- Trigger logic
```

```
END;
```

- **Example:**

sql

```
CREATE TRIGGER update_employee_salary
```

```
AFTER UPDATE ON Employees
```

```
FOR EACH ROW
```

```
BEGIN
```

```
IF OLD.Salary < NEW.Salary THEN  
    INSERT INTO SalaryHistory (EmployeeID, OldSalary, NewSalary)  
    VALUES (NEW.EmployeeID, OLD.Salary, NEW.Salary);  
END IF;  
END;
```

4. Indexes:

Indexes are used to improve query performance by allowing faster retrieval of data. They can be created on one or more columns and speed up SELECT queries. However, they can slow down INSERT, UPDATE, and DELETE operations.

- **Syntax:**

sql

```
CREATE INDEX index_name ON table_name (column1, column2);
```

- **Example:**

sql

```
CREATE INDEX idx_employee_name ON Employees (Name);
```

- **Types of Indexes:**

- **Single-column index:** Index created on a single column.
- **Composite index:** Index created on multiple columns.
- **Unique index:** Ensures that all values in a column are unique.
- **Full-text index:** Used for text searching.

10. Partitioning:

Partitioning is the technique of splitting large tables into smaller, more manageable pieces, while still treating them as a single table. This improves query performance and management of large datasets.

Steps to Create a Partitioned Table in SQL Server

1. **Define a Partition Function:** A partition function defines how the data in the table will be distributed across partitions. It specifies a column (usually a date or integer column) and the boundaries for the partitions.

2. **Define a Partition Scheme:** A partition scheme maps the partitions defined by the partition function to physical filegroups.
3. **Create the Partitioned Table:** When creating the table, you specify the partition scheme.

Example:

Let's create a partitioned table for sales data, partitioned by the Year column.

1. Create a Partition Function

First, we define how the data will be partitioned. For instance, let's partition data by Year, where each partition corresponds to a different year.

sql

Copy

```
-- Create a Partition Function that divides data into partitions by year
```

```
CREATE PARTITION FUNCTION SalesPartitionFunction (INT)
```

```
AS RANGE LEFT FOR VALUES (2020, 2021, 2022); -- Example partition values for 2020, 2021, and 2022
```

- **RANGE LEFT:** This means the partition will include the boundary value in the left partition (the previous one).
- **VALUES (2020, 2021, 2022):** The partition boundaries are specified at the years 2020, 2021, and 2022. The data before 2020 will be in the first partition, data for 2020 will be in the second partition, and so on.

2. Create a Partition Scheme

Now, we create a partition scheme that maps the partitions to physical filegroups. For simplicity, let's assume we have a filegroup PRIMARY to store the partitions.

sql

Copy

```
-- Create a Partition Scheme that uses the Partition Function
```

```
CREATE PARTITION SCHEME SalesPartitionScheme
```

```
AS PARTITION SalesPartitionFunction
```

```
TO ([PRIMARY], [PRIMARY], [PRIMARY], [PRIMARY]);
```


Here, we're mapping all partitions to the PRIMARY filegroup, but in a production environment, you may want to specify different filegroups for each partition to distribute the data across multiple disks for better performance.

3. Create the Partitioned Table

Now that the partition function and scheme are in place, we can create the table and specify the partition scheme for the table's column (e.g., SalesYear).

sql

Copy

```
-- Create a Partitioned Table
```

```
CREATE TABLE SalesData (
```

```
    SaleID INT PRIMARY KEY,
```

```
    SaleAmount DECIMAL(10, 2),
```

```
    SalesYear INT
```

```
) ON SalesPartitionScheme (SalesYear); -- Specify partition scheme on the SalesYear column
```

How It Works:

- The SalesData table is partitioned by the SalesYear column.
- Data for each year will be stored in different partitions (as defined by the partition function).
- SQL Server will handle queries on the partitioned table as if it were a single table, but it will perform operations more efficiently by focusing on the relevant partition.

Example Data Insertion:

sql

Copy

```
-- Inserting data into the partitioned table
```

```
INSERT INTO SalesData (SaleID, SaleAmount, SalesYear)
```

```
VALUES
```

```
(1, 100.50, 2020),
```

```
(2, 200.75, 2021),
```

(3, 150.60, 2022),

(4, 120.90, 2020);

Each row will be placed in the appropriate partition based on the SalesYear.

Querying Partitioned Tables:

When querying partitioned tables, SQL Server will automatically handle partition elimination, meaning it will only scan the partitions necessary for your query.

sql

Copy

-- Example query: Fetching sales data for 2021

SELECT *

FROM SalesData

WHERE SalesYear = 2021;

Benefits of Table Partitioning:

- **Improved Query Performance:** Queries that filter on the partition key (e.g., SalesYear) can be much faster because SQL Server can scan only the relevant partitions.
- **Manageability:** You can manage partitions separately. For example, you can switch partitions in and out (archive old data), rebuild indexes on specific partitions, or back up partitions independently.
- **Maintenance:** Operations like loading data, reorganizing, and rebuilding indexes can be done on individual partitions, minimizing downtime.

1/20/2025	4 PM - 5 PM	Practice Session
1/20/2025	5 PM - 6 PM	Practice Session
1/20/2025	6 PM - 7 PM	Doubt and Clarification